

SetWindowPos function (winuser.h)

10/13/2021

Changes the size, position, and Z order of a child, pop-up, or top-level window. These windows are ordered according to their appearance on the screen. The topmost window receives the highest rank and is the first window in the Z order.

Syntax

C++

```
BOOL SetWindowPos(  
    [in]          HWND hWnd,  
    [in, optional] HWND hWndInsertAfter,  
    [in]          int X,  
    [in]          int Y,  
    [in]          int cx,  
    [in]          int cy,  
    [in]          UINT uFlags  
);
```

Parameters

[in] hWnd

Type: **HWND**

A handle to the window.

[in, optional] hWndInsertAfter

Type: **HWND**

A handle to the window to precede the positioned window in the Z order. This parameter must be a window handle or one of the following values.

[Expand table](#)

Value	Meaning
HWND_BOTTOM (HWND)1	Places the window at the bottom of the Z order. If the <i>hWnd</i> parameter identifies a topmost window, the window loses its topmost status and is placed at the bottom of all other windows.

HWND_NOTOPMOST (HWND)-2	Places the window above all non-topmost windows (that is, behind all topmost windows). This flag has no effect if the window is already a non-topmost window.
HWND_TOP (HWND)0	Places the window at the top of the Z order.
HWND_TOPMOST (HWND)-1	Places the window above all non-topmost windows. The window maintains its topmost position even when it is deactivated.

For more information about how this parameter is used, see the following Remarks section.

[in] *x*

Type: **int**

The new position of the left side of the window, in client coordinates.

[in] *y*

Type: **int**

The new position of the top of the window, in client coordinates.

[in] *cx*

Type: **int**

The new width of the window, in pixels.

[in] *cy*

Type: **int**

The new height of the window, in pixels.

[in] *uFlags*

Type: **UINT**

The window sizing and positioning flags. This parameter can be a combination of the following values.

 **Expand table**

Value	Meaning
SWP_ASYNCWINDOWPOS 0x4000	If the calling thread and the thread that owns the window are attached to different input queues, the system posts the request to the thread that owns the window. This prevents the calling thread from blocking its execution while other threads process the request.
SWP_DEFERERASE 0x2000	Prevents generation of the WM_SYNCPAINT message.
SWP_DRAWFRAME 0x0020	Draws a frame (defined in the window's class description) around the window.
SWP_FRAMECHANGED 0x0020	Applies new frame styles set using the SetWindowLong function. Sends a WM_NCCALCSIZE message to the window, even if the window's size is not being changed. If this flag is not specified, WM_NCCALCSIZE is sent only when the window's size is being changed.
SWP_HIDEWINDOW 0x0080	Hides the window.
SWP_NOACTIVATE 0x0010	Does not activate the window. If this flag is not set, the window is activated and moved to the top of either the topmost or non-topmost group (depending on the setting of the <i>hWndInsertAfter</i> parameter).
SWP_NOCOPYBITS 0x0100	Discards the entire contents of the client area. If this flag is not specified, the valid contents of the client area are saved and copied back into the client area after the window is sized or repositioned.
SWP_NOMOVE 0x0002	Retains the current position (ignores <i>X</i> and <i>Y</i> parameters).
SWP_NOOWNERZORDER 0x0200	Does not change the owner window's position in the Z order.
SWP_NOREDRAW 0x0008	Does not redraw changes. If this flag is set, no repainting of any kind occurs. This applies to the client area, the nonclient area (including the title bar and scroll bars), and any part of the parent window uncovered as a result of the window being moved. When this flag is set, the application must explicitly invalidate or redraw any parts of the window and parent window that need redrawing.
SWP_NOREPOSITION 0x0200	Same as the SWP_NOOWNERZORDER flag.

SWP_NOSENDCHANGING 0x0400	Prevents the window from receiving the WM_WINDOWPOSCHANGING message.
SWP_NOSIZE 0x0001	Retains the current size (ignores the <i>cx</i> and <i>cy</i> parameters).
SWP_NOZORDER 0x0004	Retains the current Z order (ignores the <i>hWndInsertAfter</i> parameter).
SWP_SHOWWINDOW 0x0040	Displays the window.

Return value

Type: **BOOL**

If the function succeeds, the return value is nonzero.

If the function fails, the return value is zero. To get extended error information, call [GetLastError](#).

Remarks

As part of the Vista re-architecture, all services were moved off the interactive desktop into Session 0. *hwnd* and window manager operations are only effective inside a session and cross-session attempts to manipulate the *hwnd* will fail. For more information, see [The Windows Vista Developer Story: Application Compatibility Cookbook](#).

If you have changed certain window data using [SetWindowLong](#), you must call **SetWindowPos** for the changes to take effect. Use the following combination for *uFlags*: **SWP_NOMOVE** |

SWP_NOSIZE | **SWP_NOZORDER** | **SWP_FRAMECHANGED**.

A window can be made a topmost window either by setting the *hWndInsertAfter* parameter to **HWND_TOPMOST** and ensuring that the **SWP_NOZORDER** flag is not set, or by setting a window's position in the Z order so that it is above any existing topmost windows. When a non-topmost window is made topmost, its owned windows are also made topmost. Its owners, however, are not changed.

If neither the **SWP_NOACTIVATE** nor **SWP_NOZORDER** flag is specified (that is, when the application requests that a window be simultaneously activated and its position in the Z order changed), the value specified in *hWndInsertAfter* is used only in the following circumstances.

- Neither the **HWND_TOPMOST** nor **HWND_NOTOPMOST** flag is specified in *hWndInsertAfter*.

- The window identified by *hWnd* is not the active window.

An application cannot activate an inactive window without also bringing it to the top of the Z order. Applications can change an activated window's position in the Z order without restrictions, or it can activate a window and then move it to the top of the topmost or non-topmost windows.

If a topmost window is repositioned to the bottom (**HWND_BOTTOM**) of the Z order or after any non-topmost window, it is no longer topmost. When a topmost window is made non-topmost, its owners and its owned windows are also made non-topmost windows.

A non-topmost window can own a topmost window, but the reverse cannot occur. Any window (for example, a dialog box) owned by a topmost window is itself made a topmost window, to ensure that all owned windows stay above their owner.

If an application is not in the foreground, and should be in the foreground, it must call the [SetForegroundWindow](#) function.

To use **SetWindowPos** to bring a window to the top, the process that owns the window must have [SetForegroundWindow](#) permission.

Examples

For an example, see [Initializing a Dialog Box](#).

Requirements

 Expand table

Requirement	Value
Minimum supported client	Windows 2000 Professional [desktop apps only]
Minimum supported server	Windows 2000 Server [desktop apps only]
Target Platform	Windows
Header	winuser.h (include Windows.h)
Library	User32.lib
DLL	User32.dll
API set	ext-ms-win-ntuser-window-l1-1-0 (introduced in Windows 8)

See also

Conceptual

[MoveWindow](#)

Reference

[SetActiveWindow](#)

[SetForegroundWindow](#)

[Windows](#)